

Chapter 8

AI for Risk Management and Optimization

The most successful application of artificial intelligence (AI) to asset management may not be the most obvious one. Sometimes, AI should be used to correct human decisions, instead of making them autonomously à la *The Terminator*. AI can also be used for optimizing resource allocation that adapts to the changing environment. We call the former technique Corrective AI, and the latter Conditional Parameter Optimization (CPO). At [Predictnow.ai](https://predictnow.ai) and QTS Capital Management, we have put both techniques into production with significant commercial success.

What Is Corrective AI and Conditional Parameter Optimization?

The holy grail of machine learning (ML) is Artificial General Intelligence: essentially cloning a silicon version of a human being that can make all autonomous decisions. Unfortunately (or fortunately depending on your point of view), that day is still far off. For example, despite the billions of R&D dollars spent on building fully autonomous vehicles, we don't yet find too many of them driving around our neighborhoods. (For a sobering assessment of Tesla Fully Self-driving capability, check out qnt.co/book-tesla-autopilot.) On the other hand, we can hardly buy a new car these days without a fully loaded set of assisted driving technologies.

The same is true in asset management. I have spent decades, starting at IBM's Watson Lab, later at various financial institutions such as Morgan Stanley's AI group, and finally at QTS Capital Management, researching machine learning and then applying it to fully autonomous trading systems. If it were a resounding success, I would have been too busy sipping champagne on my super yacht to write about this. The reality is that few funds have found success in this endeavor (López de Prado, 2017). But a few years ago, we learned that there was a much more pragmatic way to deploy machine learning to commercial problems that does not involve waiting for decades. It is already here.

We believe that AI is much more effective in correcting erroneous decisions made by humans or other simple algorithms than in making them from scratch. That's what we call "Corrective AI."

When we learned of this concept from the financial machine-learning expert Dr. Marcos Lopez de Prado who described it as "meta-labeling" (López de Prado, 2018), we immediately tested it on our fund's crisis alpha strategy in late 2019. To our amazement, it made three consequential, and correct, predictions on that strategy. From November 2019 to January 2020, Corrective AI told us that we shouldn't expect any crisis in the market, and therefore shouldn't run our crisis alpha strategy. Investors thought we were asleep in those 3 months and grew concerned. Starting in early February 2020, it suddenly flashed red and told us to expect a crisis and recommended trading at full leverage. The subsequent COVID-induced financial crisis led to about 80% gross return in the next few months. But Corrective AI wasn't done yet. In early November 2020, it once again told us that we shouldn't expect any more crises. A few days later, on November 9, Pfizer and BioNTech announced their vaccine.

Because of the shocking effectiveness and the general applicability of this approach, we launched [Predictnow.ai](https://predictnow.ai) to commercialize this technique beyond finance. After all, asset management is the most challenging use case for AI (López de Prado, 2017). If Corrective AI worked for us in asset management with its ultra-low signal-to-noise and ultra-competitive arbitrage environment, surely it would work charmingly in other enterprise applications.

Beyond correcting decisions, we also found that AI can provide a way to optimize any business process, especially in resource allocation, more effectively than traditional optimization methods. Traditional optimization methods involve finding the parameters that generate the best results based on past data for a given business process, such as a trading strategy or stocking a retail shelf. However, when the objective function is dependent on external time-varying and stochastic conditions, traditional optimization methods may not produce optimal results. For example, in the case of a trading strategy, an optimal stop loss may depend on the market regime, which may not be clearly defined. In the case of retail shelf space optimization, the optimal selection may depend on whether it is a back-to-school season or a holiday season. Furthermore, even if the exact set of conditions is specified, the outcome may not be deterministic. Machine learning is a better alternative to solving this problem. Using machine learning, we can approximate this objective function by training its many nodes using historical data. We call this novel optimization method Conditional Parameter Optimization, or more specifically Conditional Portfolio Optimization (CPO either way), when the objective is to optimize capital allocation of a portfolio.

Encouragingly, these ideas were well-received across multiple industries. We spoke to the principals of an oil exploration firm. They used a simple formula with just a small handful of variables to predict the productivity of an oil well. We showed them how machine learning can exploit a much larger set of variables, discover their non-linear dependencies, and correct the predictions of their simple formula. Note that we are not asking

them to replace their existing formula, which is well-known and well-trusted in the oil industry—we are only correcting it.

Similarly, semiconductor manufacturers often use an expert system, handcrafted with many rules created by process engineers, to predict the outcome of a manufacturing process. We spoke to the head of production control and the head of AI of a major semiconductor manufacturer and discussed how Corrective AI can learn from a much larger set of predictors and make real-time adjustments to their manufacturing process.

In practically every enterprise, there will be an opportunity for Corrective AI and CPO to improve their process. We deliver this service to all verticals via a common Software-as-a-Service (SaaS) platform but with bespoke input features specific to each vertical.

Naturally, because of our domain expertise in finance, we have created our earliest product and found the earliest success in the asset management industry. We launched our SaaS platform for asset managers to correct their investment decisions and optimize their capital allocations, and to date our many clients range from professional traders to institutional fund managers, spanning both traditional and crypto asset classes.

New users to our platform are often surprised to find that we don't always need to use deep learning techniques. Researchers have found that deep learning, while enormously successful in image and speech recognition and other natural language processing tasks, is not well-suited to most commercial datasets (including most financial data sets). Such data are often "heterogeneous" (Shwartz-Ziv and Armon, 2021). Furthermore, deep learning tends to require a much larger dataset than is available in commercial use cases, and it often requires a much longer time to train than the technique we use. Another advantage of our technique is "explainability": we have published multiple papers on how we can offer better intuition to the human operators of what input features are really important to the predictions (Man and Chan, 2021a, 2021b). We have ultimately gone beyond merely "correcting decisions" to "improving decisions," that is, optimizing the parameters of a commercial process via CPO.

In the process of serving asset managers, we learned another key success factor in commercializing AI. No matter how good your algorithm is, it is useless to the customer unless we also provide pre-engineered input features, customized to each application domain. For asset managers, we created more than 600 such features in traditional markets (Nautiyal and Chan, 2021) and more than 800 such features in crypto markets (Viville, Sawal, and Chan, 2022). It requires our deep domain expertise to create these features, but many of our customers find them to be our most compelling value proposition. The same is true for our proof-of-concept with our oil exploration partner. They thought they had just a handful of input features to predict oil well productivity. One of our data scientists with geological engineering expertise showed them how we can create tens of thousands of them. In the next section, we will summarize some of the features used in asset management.

Feature Engineering

Features are inputs to supervised machine-learning models ([Figure 8.1](#)). In traditional finance, they are typically called “factors,” and they are used in linear regression models to either explain or predict returns. In the former usage, the factors are contemporaneous with the target returns, while in the latter the factors must be from a prior period.



[Figure 8.1](#) All these names are synonymous.

There are generally two types of factors: cross-sectional vs. time-series (Ruppert and Matteson, 2015). If you are modeling stock returns, cross-sectional factors are variables that are specific to an individual stock, such as its earnings yield, dividend yield, and so on. For example, [Predictnow.ai](#) has created 40 stocks’ fundamental features, free to all QuantConnect subscribers. These cross-sectional features are sourced from the Sharadar stock fundamental dataset. This dataset contains more than 6,000 US listed companies and nearly 10,000 delisted companies. We have carefully vetted this list to ensure it is survivorship-bias-free and that all the data is captured point-in-time, without restatements. (You might be surprised to find that some much more expensive commercial datasets from well-known vendors are *not* point-in-time, and thus cannot be used for machine learning or backtesting.) This data is available quarterly, yearly, and for trailing 12 months (these are called the three “dimensions” in Sharadar’s terminology). The original Sharadar dataset has around 140 raw data fields. We have arrowed and filtered this down to 40 essential features for users, as many of the features in the original set were redundant or highly correlated.

To use this data for machine learning, the time series data has been made stationary. For indicators that are denominated in dollars, such as total assets or capex (capital expenditure), we compute the percentage change between a given filing and the next. For indicators that are already in percentages like gross margin or net margin, we simply take the difference between the values for successive filings. These conversions are done for each of the three dimensions: quarterly, yearly and trailing 12 months.

We have also normalized this data cross-sectionally to facilitate easy comparisons across different stocks. This allows users to merge data from different stocks into a single training data set for easy input into our application program interface (API). We multiply ratios like “earnings-per-share” or “debt-per-share” with the total number of common shares outstanding, divide by the enterprise value, then finally take the difference between consecutive filings. To reiterate, this normalization is done for each of the three dimensions.

[Predictnow.ai](#) has also ensured that a reused ticker symbol is assigned to the last company that uses that ticker, not to a delisted company that previously had that ticker. This is to avoid any kind of survivorship biases. All reused ticker symbols are available with a numeric counter as a post-

fix at the end. For example, Australia Acquisition corporation and Ares Acquisition corporation both have the ticker symbol AAC. Australia Acquisition corporation was delisted in 2012, so it is available as AAC1 in our database. In contrast, Ares Acquisition corporation is still actively listed as AAC. Conversely, in the case where the ticker symbol for a company has changed over time, all its data is tied to its last valid ticker.

As Securities and Exchange Commission (SEC) filings generally happen at arbitrary times on any given day, to avoid look-ahead biases during back-tests, the date of the associated data has been shifted to the next trading day to guarantee its availability long before the market opens.

[Predictnow.ai](#) enables automatic merging of this fundamental data with your own technical or higher frequency data as one single input dataset for machine learning. When merging the Sharada fundamental features with higher frequency data, the fundamental features are forward filled until the next valid filing date. For example, gross margin for quarterly filings will be forward filled for all dates until the gross margin for the next quarter is available on the date of its filing.

But as we advocate using ML for risk management and capital allocation purposes (i.e., Corrective AI and CPO), not for returns predictions, you may wonder how these factors can help predict the returns of your trading strategy or portfolio. For example, if you have a long-short portfolio of tech stocks, such as AAPL, GOOG, AMZN, and so on, and want to predict whether the portfolio as a whole will be profitable in a certain market regime, does it really make sense to have the earnings yields of AAPL, GOOG, and AMZN as individual features? We will discuss how they can be used soon.

Meanwhile, time-series factors are typically market-wide or macroeconomic variables such as the familiar Fama-French (1995) three factors: market (simply, the market index return), SMB (the relative return of small cap vs. large cap stocks), and HML (the relative return of value vs. growth stocks). These time-series factors are eminently suitable for Corrective AI and CPO because they can be used to predict your portfolio or strategy's returns.

Given that many more obvious cross-sectional factors than time-series factors are available ([Figure 8.2](#)), it seems a pity that we cannot use cross-sectional factors as features for Corrective AI and CPO. Actually, we can; Eugene Fama and Ken French themselves showed us how. If we have a cross-sectional factor on a stock, all we need to do is to use it to rank the stocks, form a long-short portfolio using the rankings, and use the returns of this portfolio as a time-series factor. The long-short portfolio is called a hedge portfolio.

FIGURE 8.2
Cross-section vs. time-series features.

Cross-sectional Factors	Time-series Factors
Stock-specific	Market-wide
e.g., P/E, B/M, DivYld, ...	e.g., HML, SMB, WML, ...
Use that to explain/predict stocks' returns	Use that explain/predict portfolio's/strategy's returns
Regression models	Classification models

We show the process of creation of a hedge portfolio with the help of an example, starting with Sharadar’s fundamental cross-sectional factors (which we generated as shown in Nautiyal and Chan, 2021). There are 40 cross-sectional factors updated at three different frequencies: quarterly, yearly, and 12 month trailing. In this example, however, we use only the quarterly cross-sectional factors. Given a factor like capex (capital expenditure), we consider the normalized (the normalization procedure was discussed earlier) capex of approximately 8,500 stocks on particular dates from January 1, 2010, until the current date. Four particular dates are of interest every year: January 15, April 15, July 15, and October 15. We call these the ranking dates. On each of these dates we find the percentile rank of the stock based on normalized capex. The dates are carefully chosen to capture changes in the cross-sectional factors of the maximum number of stocks post the quarterly filings.

Once the capex across stocks is ranked at each ranking date (four dates) each year, we obtain the stocks present in the upper quartile (i.e., ranked above 75 percentile) and the stocks present in the lower quartile (i.e., ranked below 25 percentile). We take a long position on the ones that showed highest normalized capex and take a short position on the ones with the lowest. Both these sets together make our long-short hedge portfolio.

Once we have the portfolio on a given ranking date, we generate the daily returns of the portfolio using risk parity allocation (i.e., allocate proportional to inverse volatility). The daily returns of each chosen stock are calculated for each day till the next ranking date. The portfolio weights on each day are the normalized inverse of the rolling standard deviation of returns for a 2-month window. These weights change on a daily basis and are multiplied to the daily returns of individual stocks to get the daily portfolio returns. If a portfolio stock is delisted in between ranking dates, we simply drop the stock and do not use it to calculate the portfolio returns. The daily returns generated in this process are the capex time series factors. This process is repeated for all other Sharadar cross-sectional factors.

So, voila! Forty cross-sectional factors become forty time-series factors, and they can be used as input to Corrective AI or CPO applications for any

portfolio or trading strategy, whether it trades stocks, futures, FX, or anything at all.

Following are a number of other notable features we created:

- **NOPE** (net options pricing effect; Francus 2020) is a normalized measure of the net delta imbalance between the put and call options of a traded instrument across its entire option chain, calculated at the market close for contracts of all maturities. This indicator was invented by Lily Francus and is normalized with the total traded volume of the underlying instrument. The imbalance estimates the amount of delta hedging by market makers needed to keep their positions delta-neutral. This hedging causes price movement in the underlying, which NOPE should ideally capture. The data for this has been sourced from deltaneutral.com, and the instrument we applied it to was SPY ETF options. The SPX index options weren't used because the daily traded volume of the underlying SPX index "stock" was ill-defined. It was calculated as the traded volume of the constituents of the index.
- **Canary** is an indicator that acts similar to a canary in a coal mine and will raise an alarm when there's an impending danger. This indicator comes from the dual momentum strategies of Keller and Keuning (2017). The canary value can be either 0, 1, or 2. This is a daily measure of which of the two bond or stock ETFs has a negative absolute momentum: (1) BND, Vanguard Total Bond Market ETF, (2) VMO, Vanguard Emerging Markets Stock Index Fund ETF. The momentum is calculated using the 13612W method where we take a proportionally weighted average of percentage change in the bond/stock ETF returns in the last 1 month, 3 months, 6 months, and 1 year. In the paper, the values of "0," "1," or "2" of the canary portfolio represent what percentage of the canary is bullish. This indicates what proportion of the asset portfolio was allocated to global risky assets (equity, bond, and commodity ETFs) and what proportion was allocated to cash. For example, a "2" would imply 100% cash or cash equivalents, while a "0" would imply 100% allocation to the global risky assets. Alternatively, a value of "1" would imply 50% allocation to global risky assets and 50% to cash.
- **Carry** (Koiijen et al. 2016) defined a carry feature as "the return on a futures position when the price stays constant over the holding period." (It is also called "roll yield" or "convenience yield.") We calculate carry for (1) global equities, calculated as a ratio of expected dividend and daily close prices; (2) SPX futures, calculated from price of front month SPX futures contract and spot price of the index; and (3) currency, calculated from the two nearest months' futures data.
- **Order flow**. The underlying reason for the price movement for an asset is the imbalance of buyers and sellers. An onslaught of market sell orders portends a decrease in price and vice versa. Order flow is the signed transaction volume aggregated over a period and over many transactions in that period to create a more robust measure. It's also positively correlated with the price movement. This feature is calculated using tick data from Algoseek with aggressor tags (which flag the trade as a buy or sell market order). The data is time-stamped at mil-

liseconds. We aggregate the tick-based order flow to form order flow per minute.

Consider the following example.

Order flow feature with time stamp 10:01 am ET will consider trades from 10:00:00 am ET to 10:00:59 am ET, as shown in [Table 8.1](#).

TABLE 8.1
Example trade ticks

Time	Trade Size	Aggressor Tag
10:00:01 am	1	B
10:00:03 am	4	S
10:00:09 am	2	B
10:00:19 am	1	S
10:00:37 am	5	S
10:00:59 am	2	S

The order flow would be $1 - 4 + 2 - 1 - 5 - 5 = -9$. This would be reflect in our feature as Time:10:01, Order flow: -9

Applying Corrective AI to Daily Seasonal Forex Trading

Let us now illustrate how Corrective AI can improve the Sharpe ratio of a daily seasonal Forex trading strategy. This trading strategy takes advantage of the intraday seasonality of forex returns. Breedon and Rinaldo (2012) observed that foreign currencies depreciate versus the US dollar during their local working hours and appreciate during the local working hours of the US dollar. We first backtested the results of Breedon and Rinaldo on recent EUR/USD data from September 2021 to January 2023, and then applied Corrective AI to this trading strategy to achieve a significant increase in performance. (Portions of this section were published in Belov, Chan, Jetha, and Nautiyal, 2023).

Breedon and Rinaldo (2012) described a trading strategy that shorted EUR/USD during European working hours (3 am ET to 9 am ET, where ET denotes the local time in New York, accounting for daylight savings) and bought EUR/USD during US working hours (11 am ET to 3 pm ET). The rationale is that large-scale institutional buying of the US dollar takes place during European working hours to pay global invoices and the reverse happens during US working hours. Hence, this effect is also called the “invoice effect.”

There is some supportive evidence for the time-of-the-day patterns in various measures of the forex market like volatility (Andersen and Bollerslev, 1998), turnover (Hartmann, 1999), and return (Cornett, Schwarz, and Szakmary, 1995). Essentially, local currencies depreciate during their local working hours for each of these measures and appreciate during the working hours of the United States.

Figure 8.3 describes the average hourly return of each hour in the day over a period starting from 2019–10–01 17:00 ET to 2021–09–01 16:00 ET. It reveals the pattern of returns in EUR/USD. The return pattern in the previously described “working hours” reconciles with the hypothesis of a prevalent “invoice effect” broadly. Returns go down during European working and up during US working hours.

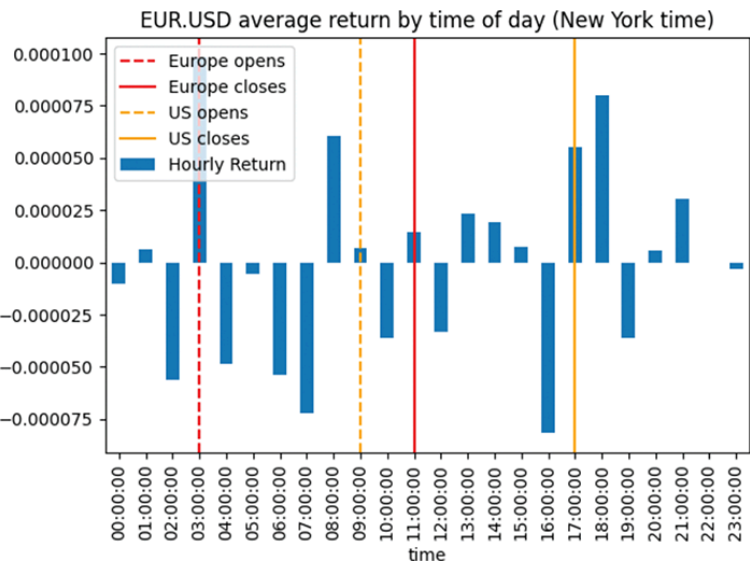


Figure 8.3 Average EURSUD return by time of day (New York time).

As this strategy was published in 2012, it offers ample time for true out-of-sample testing. We collected 1-minute bar data of EUR/USD from Electronic Broking Services (EBS) and performed a backtest over the out-of-sample period October 2021–January 2023. The Sharpe ratio of the strategy in this period is 0.88, with average annual returns of 3.5%, and a maximum drawdown of -3.5%. The alpha of the strategy apparently endured. (For the purpose of this article, no transaction costs are included in the backtest because our only objective is to compare the performances with and without Corrective AI, not to determine if this trading strategy is viable in live production.)

Figure 8.4 shows the equity curve (“growth of \$1”) of the strategy during the out-of-sample period. The cumulative returns during this period are just below 8%. We call this the “Primary” trading strategy, i.e., the strategy before any Corrective AI was applied.



Figure 8.4 Equity curve of Primary trading strategy in out-of-sample period.

The sample backtest code for this Primary trading strategy is shown in [qnt.co/bookcai-backtesting](https://github.com/quantconnect/quantconnect/blob/master/Examples/Python/BookCai/Backtesting.py). (The code cannot actually be executed without input data, and license agreement would not allow us to share the input data.)

Code for Primary Trading Strategy

```
FOREX Strategy using Corrective Artificial Intelligence (CAI)

# This notebook connects to PredictNow, trains a model, and generates
# predictions. The model hypothesis is that USD will rise against
# the EUR during EUR business hours and # all during the USD business
# hours. This is called the time of the day effect and seen due to
# HF OF and returns.

# Connect to PredictNow
from AlgorithmImports import *
from QuantConnect.PredictNowNET import PredictNowClient
from QuantConnect.PredictNowNET.Models import *
from datetime import datetime, time
from io import StringIO
import pandas as pd

qb = QuantBook()
client = PredictNowClient("account@email.com", "your_username")
client.connected

#Prepare the Data
# In this notebook, we will create a strategy that short EURUSD when
# Europe is open and long when Europe is closed and US is open. We
# will aggregate the daily return of this static strategy that is
# activate everyday, and use CAI to predict if the strategy is
# profitable for a given date. We will follow this On and Off signal
# to create a dynamic strategy and benchmark its performance.

# load minute bar data of EURUSD
symbol = qb.add_forex("EURUSD").symbol
df_price = qb.History(symbol, datetime(2020,1,1), datetime(2021,1,1)).loc[symbol]

# resample to hourly returns
minute_returns = df_price["close"].pct_change()
hourly_returns = (minute_returns + 1).resample('H').prod() - 1
df_hourly_returns = hourly_returns.to_frame()
```

```

df_hourly_returns['time'] = df_hourly_returns.index.time

# generate buy and sell signals and get strategy returns
# Sell EUR.USD when Europe is open
sell_eur = ((df_hourly_returns['time'] > time(3)) & (df_hourly_returns['time'] < time(9)))

# Buy EUR.USD when Europe is closed and US is open
buy_eur = ((df_hourly_returns['time'] > time(11)) & (df_hourly_returns['time'] < time(15)))

# signals as 1 and -1
ones = pd.DataFrame(1, index=df_hourly_returns.index, columns=['signals'])
minus_ones = pd.DataFrame(-1, index=df_hourly_returns.index, columns=['signals'])
signals = minus_ones.where(sell_eur, ones.where(buy_eur, 0))

# strategy returns
strategy_returns = df_hourly_returns['close'] * signals['signals']
strategy_returns = (strategy_returns + 1).resample('D').prod() - 1
df_strategy_returns = strategy_returns.to_frame().ffill()

# Save the Data
# We will label the data and save it to disk (ObjectStore) with the
# model name. This file will be uploaded to PredictNow.
# Define the model name and data label
model_name = "fx-time-of-day"
label = "strategy_ret"

# Label the data and save it to the object store
df_strategy_returns = df_strategy_returns.rename(columns={df_strategy_returns.columns.to_list()[0]}
parquet_path = qb.object_store.get_file_path(f'{model_name}.parquet')
df_strategy_returns.to_parquet(parquet_path)

```

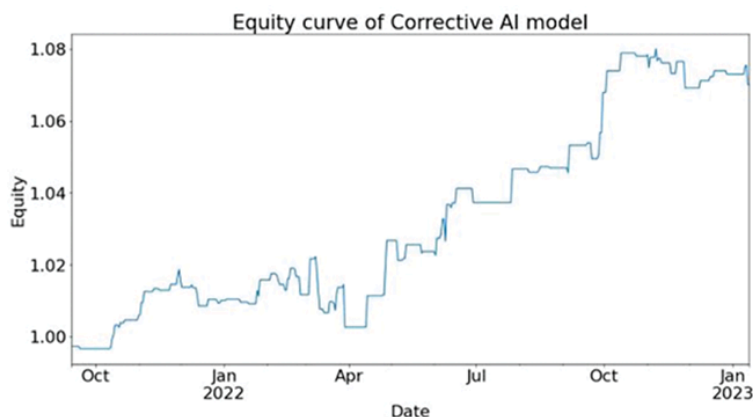
Suppose we have a trading model (like the Primary trading strategy just described) for setting the side of the bet (long or short). We just need to learn the size of that bet, which includes the possibility of no bet at all (zero sizes). This is a situation that practitioners face regularly. A machine-learning algorithm can be trained to determine that. To emphasize, we do not want the machine-learning algorithm to learn or predict the side, just to tell us what the appropriate size is. This is an application of Corrective AI because we want to build a secondary machine-learning model that learns how to use a primary trading model.

We train an machine-learning algorithm to compute the “Probability of Profit” (PoP) for the next minute-bar. If the PoP is greater than 0.5, we will set the bet size to 1; otherwise, we will set it to 0. In other words, we adopt the step function as the bet sizing function that takes PoP as an input and gives the bet size as an output, with the threshold set at 0.5. This bet sizing function decides whether to take the bet or pass, a purely binary prediction.

The training period was from 2019-01-01 to 2021-09-30 while the out-of-sample test period was from 2021-10-01 to 2023-01-15, consistent with the out-of-sample period we reported for the Primary trading strategy. The model used to train ML algorithm was done using the [predictnow.ai](https://learning-oreilly-com.ezproxy.sfpl.org/library/view/hands-on-ai-trading/9781394268436/c008.xhtml) Corrective AI API, with more than a hundred pre-engineered input features (predictors). The underlying learning algorithm is a gradient-boosted decision tree.

After applying Corrective AI, the Sharpe ratio of the strategy in this period is 1.29 (an increase of 0.41), with average annual returns of 4.1% (an increase of 0.6%) and a maximum drawdown of -1.9% (a decrease of 1.6%). The alpha of the strategy is significantly improved.

The equity curve of the Corrective AI filtered secondary model signal can be seen in [Figure 8.5](#).



[Figure 8.5](#) Equity curve of Corrective AI model in out-of-sample period.

The sample training, testing, and backtest codes for this Corrective AI-enhanced trading strategy are shown in gnt.co/book-cai-research. (The code cannot actually be executed without a Premium Subscription to [Predictnow.ai](#)'s API.)

Features used to train the Corrective AI model include technical indicators generated from indices, equities, futures, and options markets. Many of these features were created using Algoseek's high-frequency futures and equities data. More discussions of these features can be found in Nautiyal and Chan (2021).

Corrective AI Code

```
# Create the Model
# Create the model by sending the parameters to PredictNow
model_parameters = ModelParameters(
    mode=Mode.TRAIN,
    type=ModelType.CLASSIFICATION,
    feature_selection=FeatureSelection.SHAP,
    analysis=Analysis.SMALL,
    boost=Boost.GBDT,
    testsize=42.0,
    timeseries=False,
    probability_calibration=False,  # True to refine your probability
    exploratory_data_analysis=False,  # True to use exploratory analysis
    weights="no")  # yes, no, custom

create_model_result = client.create_model(model_name, model_parameters)
str(create_model_result)

# Train the Model
# Provide the path to the data, and its label. This task may take
# several minutes.
train_request_result = client.train(model_name, parquet_path, label)
```

```

str(train_request_result)
# Get the training result
# The training results include dataframes with performance metrics
# and predicted probability and labels.
training_result = client.get_training_result(model_name)
str(training_result)

# Predicted probability (float between 0 and 1) for
# validation/training data set the last column notes the probability
# that it's a "1", i.e. positive return
predicted_prob_cv = pd.read_json(StringIO(training_result.predicted_prob_cv))
print("predicted_prob_cv")
print(predicted_prob_cv)

# Predicted probability (float between 0 and 1) for
# the testing data set
predicted_prob_test = pd.read_json(StringIO(training_result.predicted_prob_test))
print("predicted_prob_test")
print(predicted_prob_test)

# Predicted label, 0 or 1, for validation/training data set.
# Classified as class 1 if probability > 0.5
predicted_targets_cv = pd.read_json(StringIO(training_result.predicted_targets_cv))
print("predicted_targets_cv")
print(predicted_targets_cv)

# Predicted label, 0 or 1, for testing data set.
# Classified as class 1 if probability > 0.5
predicted_targets_test = pd.read_json(StringIO(training_result.predicted_targets_test))
print("predicted_targets_test")
print(predicted_targets_test)

# Feature importance score, shows what features are being
# used in the prediction. More helpful when you include your features
# and only works when you set feature_selection to
# FeatureSelection.SHAP or FeatureSelection.CMDA
if training_result.feature_importance:
    feature_importance = pd.read_json(StringIO(training_result.feature_importance))
    print("feature_importance")
    print(feature_importance)

# Performance metrics in terms of accuracies
performance_metrics = pd.read_json(StringIO(training_result.performance_metrics))
print("performance_metrics")
print(performance_metrics)
# Start Predicting with the Trained Model
predict_result = client.predict(model_name, parquet_path, exploratory_data_analysis=False, probabi
str(predict_result)

```

By applying Corrective AI to the time-of-the-day Primary strategy, we were able to improve the Sharpe ratio and reduce drawdown during the out-of-sample backtest period. This aligns with observations made in the literature on meta-labeling.

What Is Conditional Parameter Optimization?

Every trader knows that there are market regimes that are favorable to their strategies and others that are not. Some regimes are obvious, like

bull vs. bear markets, calm vs. choppy markets, and so forth. These regimes affect many strategies and portfolios (unless they are market-neutral or volatility-neutral portfolios) and are readily observable and identifiable (but perhaps not predictable). Other regimes are more subtle and may only affect a specific strategy. Regimes may change every day, and they may not be observable. It is often not as simple as saying the market has two regimes, and we are currently in regime 2 instead of 1. For example, with respect to the profitability of your specific strategy, the market may have an infinite number of regimes ranging from very favorable to very unfavorable. For example, a momentum trading strategy's returns could be positively correlated with market volatility, which is obviously a continuous, not a discrete, variable.

Regime changes sometimes necessitate a complete change of trading strategy (e.g., trading a mean-reverting instead of momentum strategy). Other times, traders just need to change the parameters of their existing trading strategy to adapt to a different regime. As we mentioned earlier, [PredictNow.ai](#) has come up with a novel way of adapting the parameters of a trading strategy called CPO. This invention allows traders to adapt their trading parameters as frequently as they like—perhaps for every trading day or even every single trade. (This section is substantially the same as that in [Chapter 7](#) of Chan 2021 and is reprinted here for completeness and ease of reference.) CPO uses machine learning to place orders optimally based on changing market conditions (regimes) in any market. Traders in these markets typically already possess a basic trading strategy that decides the timing, pricing, type, and/or size of such orders. This trading strategy will usually have a small number of adjustable trading parameters. Conventionally, they are often optimized based on a fixed historical data set (“train set”). Alternatively, they may be periodically re-optimized using an expanding or rolling train set. (The latter is often called “Walk Forward Optimization.”) With a fixed train set, the trading parameters clearly cannot adapt to changing regimes. With an expanding train set, the trading parameters still cannot respond to rapidly changing market conditions because the additional data is but a small fraction of the existing train set. Even with a rolling train set, there is no evidence that the parameters optimized in the most recent historical period generate better out-of-sample performance. A too-small rolling train set will also give unstable and unreliable predictive results, given the lack of statistical significance. All these conventional optimization procedures can be called unconditional parameter optimization, as the trading parameters do not intelligently respond to rapidly changing market conditions. Ideally, we would like trading parameters that are much more sensitive to the market conditions and yet are trained on a large enough amount of data.

To address this adaptability problem, we apply a supervised machine-learning algorithm (we have used random forest with boosting, but the CPO methodology is indifferent to the specific learning algorithm) to learn from a large feature set that captures various aspects of the prevailing market conditions, together with specific values of the trading parameters, to predict the outcome of the trading strategy. (An example outcome is the strategy's future one-day return.) Once such machine-learn-

ing model is trained to predict the outcome, we can apply it to live trading by feeding in the same features that represent the latest market conditions as well as various combinations of the trading parameters. The set of parameters that results in the optimal predicted outcome (e.g., the highest future one-day return) will be selected as optimal and will be adopted for the trading strategy for the next period. The trader can make such predictions and adjust the trading strategy as frequently as needed to respond to rapidly changing market conditions.

In the example in the next section, we apply CPO using [PredictNow.ai](#)'s financial machine-learning API to adapt the parameters of a Bollinger Band-based mean reversion strategy on GLD (the gold ETF) and obtain superior results.

The CPO technique is useful in industry verticals other than finance as well; after all, optimization under time-varying and stochastic condition is a very general problem. For example, wait times in a hospital emergency room may be minimized by optimizing various parameters, such as staffing level, equipment and supplies readiness, discharge rate, and so on. Current state-of-the-art methods generally find the optimal parameters by looking at what worked best on average in the past. There is also no mathematical function that exactly determines wait time based on these parameters. The CPO technique can employ other variables, such as time of day, day of week, season, weather, whether there are recent mass events, and so on, to predict the wait time under various parameter combinations, and thereby find the optimal combination under the current conditions to achieve the shortest wait time.

Applying Conditional Parameter Optimization to an ETF Strategy

To illustrate the CPO technique, we next describe an example trading strategy on an ETF.

This strategy uses the lead-lag relationship between the GLD and GDX ETFs using 1-minute bars from January 1, 2006, until December 31, 2020, splitting it 80%/20% between train/test periods. The trading strategy has three trading parameters: the hedge ratio (`GDX_weight`), entry threshold (`entry_threshold`), and a moving lookback window (`lookback`). The spread is defined as follows:

$$\text{Spread } t = \text{GLD_close } t - \text{GDX_close } t * \text{GDX_weight.} \quad (1)$$

We may enter a trade for GLD at time t , and exit it at time $t + 1$ minute, hopefully realizing a profit. We want to optimize the three trading parameters on a $5 \times 10 \times 8$ grid. The grid is defined as follows:

`GDX_weight` = {2, 2.5, 3, 3.5, 4}

`entry_threshold` = {0.2, 0.3, 0.4, 0.5, 0.7, 1, 1.25, 1.5, 2, 2, 5}

`lookback` = {30, 60, 90, 120, 180, 240, 360, 720}

To be clear, even though we are using GLD and GDX prices and functions of these prices to make trading decisions, for simplicity of illustration we only trade GLD, unlike the typical long-short pair trading setup.

Every minute we compute $\text{Spread}(t)$ in [equation \(1\)](#), and compute its “Bollinger Bands,” conventionally defined as follows:

$$Z_score(t) = \frac{\text{Spread}(t) - \text{Spread_EMA}(t)}{\sqrt{\text{Spread_VAR}(t)}} \quad (2)$$

where Spread_EMA is the exponential moving average of the Spread, and Spread_VAR is its exponential moving variance (see Definitions of Spread_EMA & Spread_VAR at the end of this chapter for their conventional definitions).

Similar to a typical mean-reverting strategy using Bollinger Bands, we trade into a new GLD position based on these rules:

1. Buy GLD, if $Z_score < -\text{entry_threshold}$ (resulting in long position).
2. Short GLD, if $Z_score > \text{entry_threshold}$ (resulting in short position).
3. Liquidate long position, if $Z_score > \text{exit_threshold}$.
4. Liquidate short position, if $Z_score < -\text{exit_threshold}$.

The exit_threshold can be anywhere between entry_threshold and $-\text{entry_threshold}$. After optimization in the train set, we set $\text{exit_threshold} = -0.6 * \text{entry_threshold}$ and keep that relationship fixed when we vary entry_threshold in our future (unconditional or conditional) parameter optimizations. We trade the strategy on 1-minute bars between 9:30 and 15:59 ET, and liquidate any position at 16:00. For each combination of our three trading parameters, we record the daily return of the resulting intraday strategy and form a time series of daily strategy returns, to be used as labels for our machine-learning step in CPOs. Note that since the trading strategy may execute multiple round trips per day before forced liquidation at the market close, this daily strategy return is the sum of such round-trip returns.

Unconditional vs. Conditional Parameter Optimizations

In conventional, unconditional, parameter optimization, we select the three trading parameters (GDX_weight , entry_threshold , and lookback) that maximize cumulative in-sample return over the three-dimensional parameter grid using exhaustive search. (Gradient-based optimization did not work due to multiple local maxima). We use that fixed set of three optimal trading parameters and use them to specify the strategy out-of-sample on the test set.

With conditional, parameter optimization, the set of trading parameters used each day depends on a predictive machine-learning model trained on the train set. This model will predict the future one-day return of our trading strategy, given the trading parameters and other market conditions. Since the trading parameters can be varied at will (i.e., they are control variables), we can predict a different future return for many sets

of trading parameters each day, and select the optimal set that predicts the highest future return. That optimal parameter set will be used for the trading strategy for the next day. This step is taken after the current day’s market close and before the market open of the next day.

In addition to the three trading parameters, the predictors (or “features”) for input to our machine-learning model are eight technical indicators obtained from the Technical Analysis Python library: Bollinger Bands Z-score, Money Flow, Force Index, Donchian Channel, Average True Range, Awesome Oscillator, and Average Directional Index. We choose these indicators to represent the market conditions. Each indicator actually produces 2×7 features, since we apply them to each of the ETFs GLD and GDX price series, and each was computed using seven different lookback windows: 50, 100, 200, 400, 800, 1600, and 3200 minutes. (Note: This is not the same as the trading parameter “lookback” described earlier.) Hence, there are a total of $3 + 8 \times 2 \times 7 = 115$ features used in predicting the future 1-day return of the strategy. But because there are $5 \times 10 \times 8 = 400$ combinations of the three trading parameters, each trading day comes with 400 rows of training data that looks something like [Table 8.2](#) (labels—future returns—are not displayed):

TABLE 8.2
An 1-day slice of features table as input to the machine-learning model.

GDX_weight	entry_threshold	lookback	Z-score- GLD(50)	Z-score- GDX(50)	Money- Flow- GLD(50)	Money- Flow- GDX(50)	...
2	0.2	30	0.123	0.456	1.23	4.56	...
2	0.2	60	0.123	0.456	1.23	4.56	...
2	0.2	90	0.123	0.456	1.23	4.56	...
...							
4	5	240	0.123	0.456	1.23	4.56	...
4	5	360	0.123	0.456	1.23	4.56	...
4	5	720	0.123	0.456	1.23	4.56	

After the machine-learning model is trained, we can use it for live predictions and trading. Each trading day after the market closes, we prepare an input vector, which is structured like one row of [Table 8.2](#), populated with one particular set of the trading parameters and the current values of the technical indicators, and use the machine-learning model to predict the trading strategy’s return on the next day. We do that 400 times, varying the trading parameters, but obviously not the technical indicators’ values, and find out which trading parameter set predicts the highest re-

turn. We adopt that optimal set for the trading strategy next day. In mathematical terms,

$$\begin{aligned} & GDX_weight_optimal, entry_threshold_optimal, lookback_optimal) \\ & = \operatorname{argmax}_{GDX_weight, entry_threshold, lookback} \{ \dots \\ & \quad \text{predict } GDX_weight, entry_threshold, lookback, \text{technicalindicators} \} \end{aligned} \quad (3)$$

where predict is the predictive function available from predictnow.ai's API, which uses random forest with boosting as the machine-learning model.

It is important to understand that unlike a naïve application of machine learning to predict GLD's one-day return using technical indicators, we are using machine learning to predict the return of a trading strategy applied to GLD given a set of trading parameters and using those predictions to optimize these parameters on a daily basis. The naïve approach is less likely to succeed because everybody is trying to predict GLD's (i.e., gold's) returns and inviting arbitrage activities, but nobody is predicting the returns of this particular GLD trading strategy (unless they take this toy example too seriously!). Furthermore, many traders do not like using machine learning as a black box to predict returns. In CPO, the trader's own strategy is making the actual predictions. Machine learning is merely used to optimize the parameters of this trading strategy. This provides for much greater degree of transparency and interpretability.

Performance Comparisons

We compare out-of-sample test set performance of unconditional vs. conditional parameter optimization on the last three years of data ending on December 31, 2020, and find the cumulative three-year return to be 73% and 83%, respectively. All other metrics are improved using conditional parameter optimization. See [Table 8.3](#) and [Figure 8.6](#).

TABLE 8.3

Out-of-sample performances of unconditional vs. conditional parameter optimization

	Unconditional Optimization	Conditional Optimization
Annual Return	17.29%	19.77%
Sharpe Ratio	1.947	2.325
Calmar Ratio	0.984	1.454

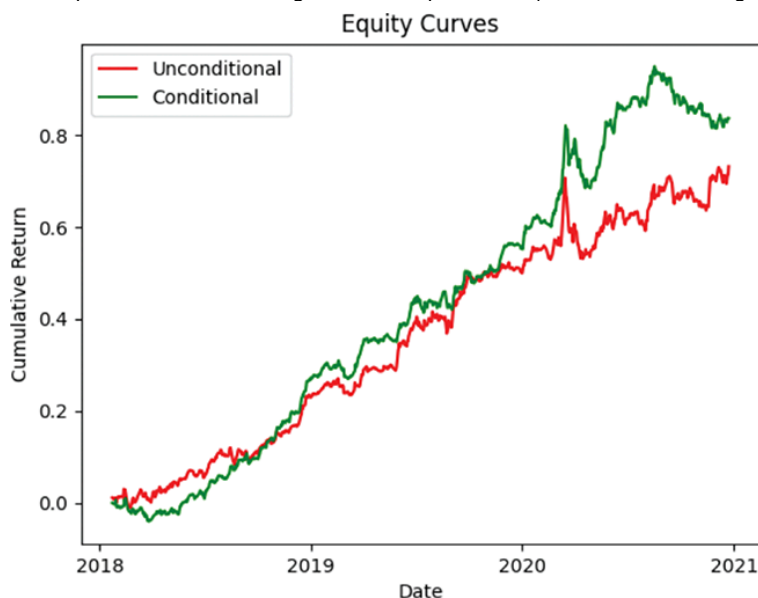


Figure 8.6 Cumulative returns of strategies based on conditional vs. unconditional parameter optimization.

Conditional Portfolio Optimization

Regime Changes Obliterate Traditional Portfolio Optimization Methods

While CPO showed promise in optimizing the operating parameters of trading strategies, its greatest potential lies in its potential to optimize *portfolio allocations*. We refer to this approach as Conditional **Portfolio** Optimization (which, fortuitously, shares the same acronym).

To recap, traditional optimization methods involve finding the parameters that generate the best results based on past data for a given business process, such as a trading strategy. For instance, a stop loss of 1% may have yielded the best Sharpe ratio for a trading strategy tested over the last 10 years, or stocking a retail shelf with sweets generate the best profit over the last 5 years. Although the numerical optimization process can be complex due to various factors, such as the number of parameters, the non-linearity of objective functions, or the number of constraints on the parameters, standard methods are available to handle such difficulties. However, when the objective function is dependent on external time-varying and stochastic conditions, traditional optimization methods may not produce optimal results. Machine learning is a better alternative to solve this problem.

In the context of financial portfolio optimization, where the parameters to optimize are the capital allocations to various components of a portfolio to achieve the best financial performance objective, there are several competing machine learning approaches. As far as we know, all of them (e.g., Tomasz and Katarzyna, 2021; or Cong et al., 2021) involve using cross-sectional features to predict the returns of individual components of a portfolio. This is a highly challenging task with uncertain success. More problematically, such cross-sectional features are also often unavailable for portfolios with components whose identities are undisclosed in commercial applications, rendering such cross-sectional returns pre-

dictions even more difficult. We will elaborate on this advantage of CPO in the following section. (Portions of this section were published in Chan, Fan, Sawal, and Ville, 2023.)

Learning to Optimize

Machine learning (especially neural networks, see Hornik, Stinchcombe, and White, 1989) can be used to approximate any function, including the objective function for our portfolio optimization, by training on historical data. (We will henceforth refer to this machine-learning model as the ML model. We primarily use gradient-boosted trees, but neural networks or other algorithms can be equally effective.) The inputs to this ML model will not only include the parameters that we originally set out to optimize, but also the vast set of features that measure the external conditions. For example, to represent a “market regime” suitable for portfolio optimizations, we may include market volatility, behaviors of different market sectors, macroeconomic conditions, and many other input features. The output of this ML model would be the outcome you want to optimize. For example, maximizing the future 1-month Sharpe ratio of a portfolio is a typical outcome. In this case, you would feed historical training samples to the ML model that include the capital allocations of the various components of the portfolio, the market features, plus the resulting forward 1-month Sharpe ratio of the portfolio as “labels” (i.e., target variables). Once trained, this ML model can then predict the future 1-month Sharpe ratio based on any hypothetical set of capital allocations and the current market features. The components of the portfolio could be stocks in a mutual fund or trading strategies in a hedge fund. For example, in [Table 8.4](#), we display the input features for one day of a portfolio optimization problem:

TABLE 8.4

A 1-day slice of features table as input to the neural network

GOOG	MSF	AAPL	VIX	Oil 30d Return	GDP Growth	VIX	...
20%	60%	20%	15.3	0.456%	1.23%	4.56%	...
25%	60%	15%	15.3	0.456%	1.23%	4.56%	...
30%	60%	10%	15.3	0.456%	1.23%	4.56%	...
...							
40%	50%	10%	15.3	0.456%	1.23%	4.56%	...
45%	50%	5%	15.3	0.456%	1.23%	4.56%	...
50%	50%	0%	15.3	0.456%	1.23%	4.56%	...

Control Features

Market Features

Assuming that the features that measure market regimes (denoted “Market features” in [Table 8.4](#)) have a daily frequency, a 1-day slice of the features table nevertheless contains many rows (samples). Each row represents a unique capital allocation—we call them “control features.” For a portfolio that holds S&P 500 stocks, for instance, there will be up to 500 parameters (if cash is included). In this case, we are supposed to feed into the neural network *all possible combinations* of these 500 parameters, plus the market features, and find out what the resulting forward 1-month Sharpe ratio (or whatever performance metric we want to maximize) is. *All possible combinations*? If we represent the capital weight allo-

cated to each stock as $w_i \in [0, 1]$, assuming we are not allowing short positions, the search space has $0, 1]^{500}$ combinations. Even discretizing to a grid search, our computer will need to run till the end of time to finish. And that is just for one day; training the neural net will require many days of such samples that include the features and the resulting labels (forward 1-month Sharpe ratio). Overcoming this curse of dimensionality by intelligently sampling the grid is one of the major breakthroughs [Predictnow.ai](#) has accomplished. Intelligent sampling involves, for example, not sampling those parts of the 500-dimensional grid that are unlikely to generate optimal portfolios, or not sampling those parts of the grid that result in portfolios that are too similar to an existing sample.

Perhaps the following workflow diagrams ([Figures 8.7, 8.8, and 8.9](#)) can illuminate the process:

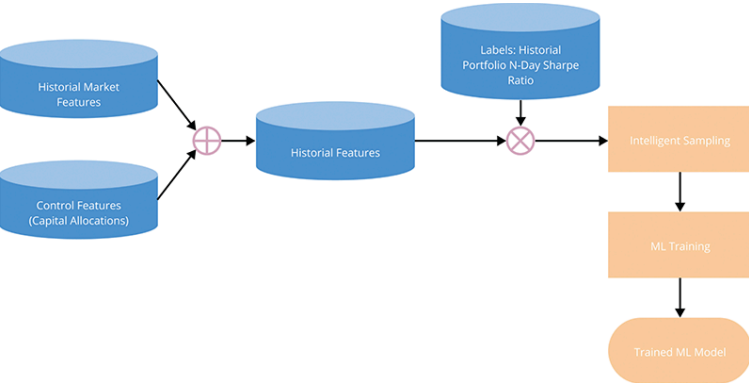


Figure 8.7 Training a portfolio performance prediction machine.

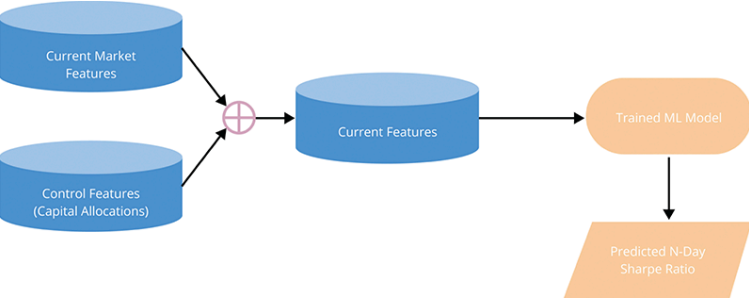


Figure 8.8 Live prediction of portfolio performance (inferences).

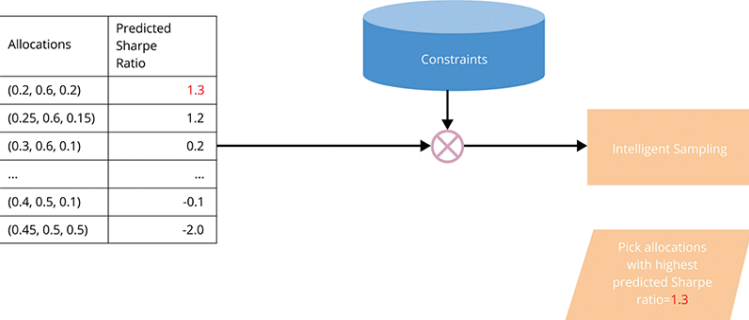


Figure 8.9 Optimization step.

Ranking Is Easier Than Predicting

Some readers may argue that if a ML model can predict the Sharpe ratio of a portfolio based on its parameters and market features, why not use

the model to directly predict the underlying assets' returns and replace the original portfolio altogether? However, with our CPO method, you don't need to predict the portfolio's Sharpe ratio accurately; you just need to predict which set of parameters give the *best* Sharpe ratio. Even if the Sharpe ratio predictions come with huge error bars, it is the *ranking* of predicted Sharpe ratio that matters. The situation is analogous to many alpha models for stock selection: to form a profitable long-short portfolio of stocks, an alpha model doesn't need to predict the cross-sectional returns accurately, but only needs to rank them correctly. We don't have an alpha model to predict or even rank such cross-sectional returns. (If we did, we wouldn't be offering that to clients as a technology product; we would offer it as a performance-fee-based managed account service.) Our client has used its own alpha model to select the portfolio components for us already. All we need to do is to rank the various capital allocations applied to a client's portfolio based on their predicted Sharpe ratios. It is a much less demanding predictive model if we only care that it gets the ranking of the labels correctly (Poh et al., 2020).

The Fama-French Lineage

With CPO, not only is it unnecessary to predict cross-sectional returns, but we also don't need to use any cross-sectional features as input to our optimizer. Instead, we only use market features that are sometimes called "time series factors" (Ruppert and Matteson, 2015). Can a ML model really predict portfolio returns without any cross-sectional features? Let's draw an analogy with the Fama-French three-factor model (Fama and French, 1995). Recall that Fama and French proposed that we can explain (not predict) a portfolio's returns using just three factors: the market index return, SMB which measures the outperformance of small cap stocks over large cap stocks, and HML, which measures the outperformance of value stocks over growth stocks. (Note that these factors can be negative, where "outperformance" becomes "underperformance.") The explanatory model is just a linear regression fit using these three factors as independent variables against the current-period portfolio return, hopefully with a decent R^2 . But if we use these as predictive factors (or features) to forecast the next-period portfolio return, the R^2 of such a regression fit will be poor. This may be because we have too few factors. We can try to improve it by adding hundreds of factors (e.g., using our "factor zoo"; Nautiyal and Chan, 2021), capturing all manners of market regimes and risks. But many of these factors will be collinear or insignificant and will continue to cause high bias and variance (Murphy, 2012). So finally, we are led to the application of non-linear machine-learning model that can deal with such multicollinearity and insignificance, via standard techniques such as features selection (Man and Chan, 2021a, 2021b) and regularization. If we also add to the input "control features" that *condition* the prediction on the capital allocation in the portfolio, we have come full circle and arrive at Conditional Portfolio Optimization.

Comparison with Conventional Optimization Methods

To assess the value of Conditional Portfolio Optimization, we need to compare it with alternative portfolio optimization methods. The default

method is Equal Weights, which involves allocating equal capital to all portfolio components. Another simple method is Risk Parity, where the capital allocation to each component is inversely proportional to its return volatility. It is called Risk Parity because each component is supposed to contribute an equal amount of risk, as measured by volatility, to the overall portfolio's risk. This method assumes zero correlations among the components' returns, which is of course unrealistic. Then there is the Markowitz method, also known as Mean-Variance optimization. This well-known method, which earned Harry Markowitz a Nobel prize, maximizes the Sharpe ratio of the portfolio based on the historical means and covariances of the component returns through a quadratic optimizer. The optimal portfolio that has the maximum historical Sharpe ratio is also called the *tangency portfolio*. One of us wrote about this method in a previous blog post (Chan, 2014) and its equivalence to the Kelly formula. It certainly doesn't take into account market regimes or any market features. It is also a vagrant violation of the familiar refrain, "Past performance is not indicative of future results," and is known to be highly sensitive to slight variations of input and to produce all manners of unfortunate instabilities (see Ang, 2014; López de Prado, 2020). Nevertheless, it is the standard portfolio optimization method that many asset managers use. Finally, there is the Minimum Variance portfolio, which uses Markowitz's method not to maximize the Sharpe ratio, but to minimize the variance (and hence volatility) of the portfolio's returns. Even though this approach does not maximize a portfolio's past Sharpe ratio, it often achieves better forward Sharpe ratios than the tangency portfolio! Another case of "past performance is not indicative of future results."

Some researchers compute expected cross-sectional returns using an alpha model, and then use Markowitz's optimization by inputting these returns (Tomasz and Katarzyna, 2021). However, in practice most alpha models do not produce expected cross-sectional returns accurately enough as input for a quadratic optimizer. As we explained before, the beauty of our method is that we don't need cross-sectional returns nor cross-sectional features of any kind as input to the ML model. Only "time series" market features are used.

Let's see how our Conditional Portfolio Optimization method stacks up against these conventional methods.

Based on a client's request, we tested how our CPO performed for an ETF (TSX: MESH) given the constraints that we cannot short any stock, and the weight $w_s \in [0.5\%, 10\%]$, but we can allocate a maximum of $w_c = 10\%$ of the portfolio to cash, with $\sum_s w_s + w_c = 1$. See [Table 8.5](#).

TABLE 8.5
MESH ETF

Period	Method	Sharpe Ratio	CAGR
2021-08–2022-07 (Out-of-sample)	Equal Weights	−0.76	−30.6%
	Risk Parity	−0.64	−22.2%
	Markowitz	−0.94	−30.8%
	Minimum Variance	−0.47	−14.5%
	CPO	−0.33	−13.7%

In the bull market, CPO performed similarly to the Markowitz method. However, it was remarkable that CPO was able to switch to defensive positions and outperformed the Markowitz method in the bear market of 2022. Overall, it improved the Sharpe ratio of the Markowitz portfolio by more than 60%. That is the whole rationale of *Conditional* Portfolio Optimization: it adapts to the expected future external conditions (market regimes) instead of blindly optimizing on what happened in the past. Because of the long-only constraint and the tight constraint on cash allocation, the CPO portfolio still suffered negative returns. But if we had allowed the optimizer to allocate a maximum of 50% of the portfolio NAV to cash, it would have delivered positive returns. The dramatic effect of cash allocation will be evident in the next example.

In the next example, we tested the CPO methodology on a private investor’s tech portfolio, consisting of seven US and two Canadian stocks, mostly in the tech sector. We call this the Tech Portfolio. The constraints are that we cannot short any stock, and the weight w_s of each stock s obeys $w_s \in [0\%, 25\%]$, and we can allocate a maximum of $w_c = 50\%$ of the portfolio to cash, with $\sum_s w_s + w_c = 1$. See Table 8.6.

TABLE 8.6
Tech Portfolio

Period	Method	Sharpe Ratio	CAGR
2021-08–2022-07 (Out-of-sample)	Equal Weights	0.39	6.36%
	Risk Parity	0.49	7.51%
	Markowitz	0.40	6.37%
	Minimum Variance	0.23	2.38%
	CPO	0.70	11.0%

CPO performed better than both alternative methods under all market conditions. It improves the Sharpe ratio over the Markowitz portfolio by 75% as the market experienced a regime shift around January 2022.

Figure 8.10 shows the comparative equity curves.



Figure 8.10 Comparative performances of various portfolio optimization methods on Tech Portfolio. (Out-of-sample period starts August 2021.)

Even though this portfolio is tech-heavy, it was able to generate a positive return during this trying out-of-sample period of 2021-08–2022-07. The reason is that it can allocate 50% of the NAV to cash, as one can see by looking at the time evolution of the cash component in Figure 8.11.

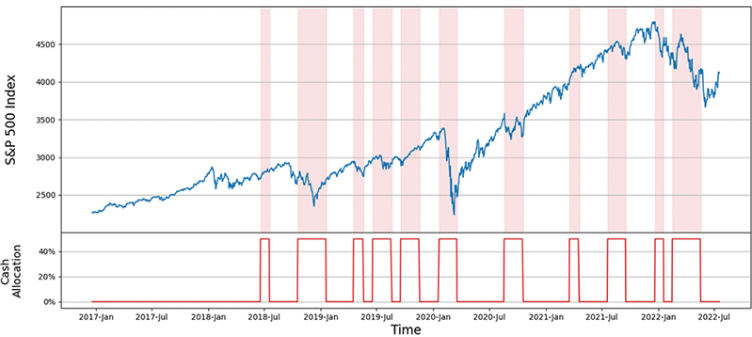


Figure 8.11 Cash allocation vs. market regime of tech portfolio.

In Figure 8.11, the highlighted time periods indicate when CPO allocated maximally to cash. The overlay of the S&P 500 Index reveals that these periods are highly correlated with the drawdown periods of the market index, even during out-of-sample period. This supports our hypothesis that CPO can rapidly adapt to market regime changes.

We also tested how CPO performs for some non-traditional assets: a portfolio of eight crypto currencies, again allowing for short positions and aiming to maximize its 7-day forward Sharpe ratio. See Table 8.7.

TABLE 8.7

Crypto portfolio

Method	Sharpe Ratio
Markowitz	0.26
CPO	1.00

(These results are over an out-of-sample period from January 2020 to June 2021, and the universe of cryptocurrencies for the portfolio are BTCUSDT, ETHUSDT, XRPUSDT, ADAUSDT, EOSUSDT, LTCUSDT, ETCUSDT, XLMUSDT). CPO improves the Sharpe ratio over the Markowitz method by a factor of 3.8.

Finally, to illustrate that CPO doesn’t just work on portfolios of assets, we apply it to a portfolio of FX trading strategies managed by a FX hedge fund WSG; see [Table 8.8](#). (WSG is our client, and we published these results with their permission.) It is a portfolio of seven trading strategies s , and the allocation constraints are $w_s \in [0\%, 40\%]$, $w_c \in [0\%, 100\%]$, with $\sum_s w_s + w_c = 1$. See [Table 8.8](#).

TABLE 8.8

WSG’s FX strategies portfolio

Method	Sharpe Ratio
Equal Weights	1.44
Markowitz	2.22
CPO	2.65

(These results are over an out-of-sample period from January 2020 to July 2022.) CPO improves the Sharpe ratio over the Markowitz method by 19%. WSG has decided to deploy CPO in production starting July 2022. Since then, CPO has added about 60bps per month to the portfolio over their previous proprietary allocation method.

In all four cases, CPO outperformed both the naive Equal Weights portfolio and the Markowitz portfolio during a market downturn, while generating similar performance during the bull market. It is important to note that we do not claim CPO can outperform *all* other allocation methods for *all* portfolios in *all* periods. Some portfolios may be constructed to be so factor-neutral that CPO can’t improve on any conventional allocation method. For other portfolios, CPO may underperform a conventional allocation method for a certain period with the benefit of hindsight (*ex post*), but nevertheless outperform the best conventional allocation method selected at the beginning of the period (*ex ante*). We provide an illustration of this effect through the following model portfolio.

Model Tactical Asset Allocation Portfolio

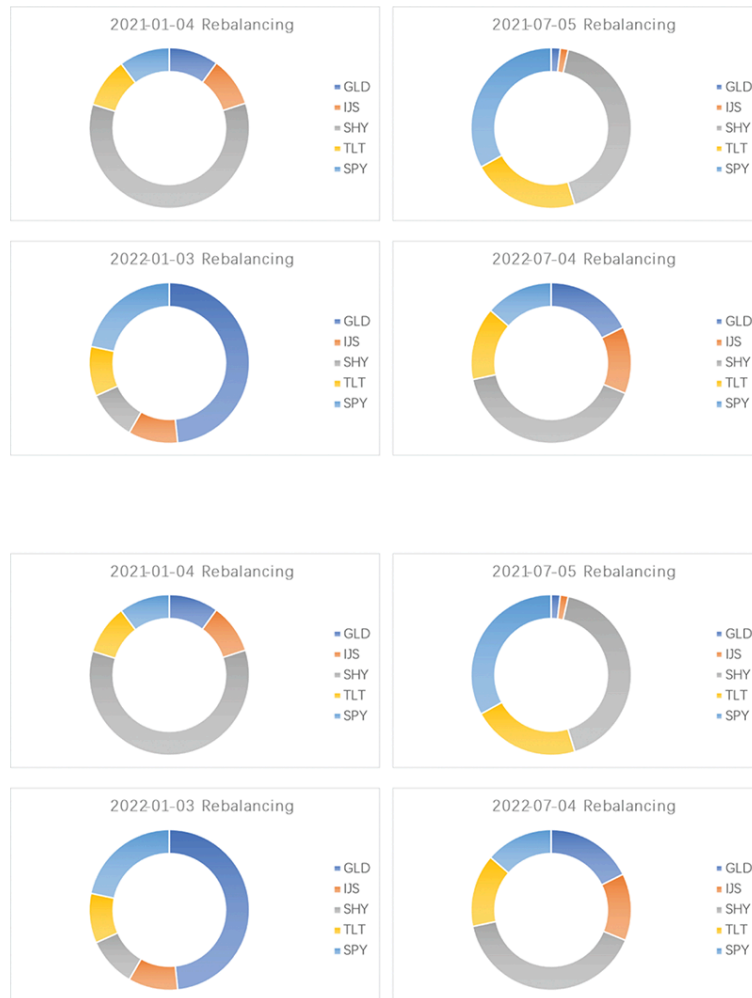
The purpose of studying a model tactical asset allocation (TAA) portfolio is not only to investigate whether CPO can outperform conventional allocation methods, but also to observe how CPO allocates across different asset classes over evolving market regimes. The model portfolio we selected comprises five ETFs representing various asset classes: GLD (gold), IJS (small cap stocks), SPY (large cap stocks), SHY (1–3 year Treasury bonds), and TLT (20+ year Treasury bonds). This portfolio is inspired by the Golden Butterfly portfolio created by (Tyler, 2016). To train our ML model, we use the period January 2015 to December 2018, while the out-of-sample test period covers January 2019 to December 2022. The portfolio rebalances every 2 weeks, and CPO aims to maximize the Sharpe Ratio over the forward 2-week period. The constraint is $w_s \in [0\%, 100\%]$, $\sum_s w_s = 1$, with no cash allocation allowed (since SHY is practically cash).

Table 8.9 shows during the out-of-sample period, CPO generates the second-highest Sharpe ratio, trailing only the Equal Weights method. However, selecting Equal Weights ex ante would not have been an obvious choice, since it generated the second-lowest Sharpe ratio during the in-sample period. If we were to choose a conventional allocation method ex ante, Risk Parity would have been our choice, but it underperformed CPO out-of-sample as measured by both the Sharpe ratio and CAGR, the latter by more than threefold.

TABLE 8.9
In- and Out-of-sample performance of TAA portfolio

Period	Method	Sharpe Ratio	CAGR
2015-01–2018-12 (In-sample)	Equal Weights	0.51	3.60%
	Risk Parity	0.62	1.87%
	Markowitz	0.59	5.26%
	Minimum Variance	0.47	1.13%
	CPO	0.63	3.93%
Period	Method	Sharpe Ratio	CAGR
2019-01–2022-12 (Out-of-sample)	Equal Weights	0.62	6.61%
	Risk Parity	0.22	1.16%
	Markowitz	-0.13	-2.09%
	Minimum Variance	-0.05	0.39%
	CPO	0.42	3.83%

To gain more transparency into the CPO method, we can examine its allocations at various times in [Figure 8.12](#).



[Figure 8.12](#) Time evolution of allocations of TAA portfolio (GLD is deep blue.)

It is noteworthy that the portfolio had a high allocation to large-cap stocks beginning in July 2019, just before the market experienced a period of calm appreciation over the next 6 months. The high allocation to short-term treasuries in January 2020 proved to be prescient in light of the COVID-induced financial crisis that followed. The portfolio also had a high allocation to gold at the beginning of 2022, which fortuitously anticipated the surge in commodity prices due to the war in Ukraine. Finally, the allocation to small-cap stocks increased in mid-2022, which performed better than large-cap stocks during that year.

CPO Software-as-a-Service

For clients of [Predictnow.ai](#)'s CPO Software-as-a-Service (SaaS) platform, we can optimize any objective function, not just Sharpe ratio. For example, we have been asked to minimize Expected Shortfall and UPI. We can also add specific constraints to the desired optimal portfolio, such as average ESG rating, maximum exposure to various sectors, or maximum turnover during portfolio rebalancing. The only other input we require is the historical returns of the portfolio components (unless these components are publicly traded assets, in which case clients only need to tell us

their tickers). If these components changed over time, we will also need the historical components.

We will provide pre-engineered market features (Nautiyal and Chan, 2021) that capture market regime information. If the client has proprietary market features that may help predict the returns of their portfolio, they can merge those with ours as well. Clients' features can remain anonymized. We will be providing an API for clients who wish to experiment with various constraints and hyperparameters (such as the frequency of portfolio rebalancing) and their effects on the optimal portfolio.

In the two code examples below, we show a sample client-side Jupyter Notebook and calls on our CPO API. (The code cannot actually be executed without input data, and license agreement would not allow us to share the input data.) (qnt.co/book-cpo-research) (qnt.co/book-cpo-backtesting).

CPO Code

```
# Generate Portfolio Weights to Run a LEAN Backtest
# This notebook connects to PredictNow, optimizes the portfolio
# weights for each rebalance, and then saves the rebalancing weights
# for each month into the Object Store. After you run the cells in
# this notebook, you can run the algorithm in main.py, which uses
# the portfolio weights from PredictNow in a LEAN backtest.

# Connect to PredictNow

from QuantConnect.PredictNowNET import PredictNowClient
from QuantConnect.PredictNowNET.Models import *
from time import sleep
from datetime import datetime
algorithm_start_date = datetime(2020, 2, 1)
algorithm_end_date = datetime(2024, 4, 1)

qb = QuantBook()
client = PredictNowClient("test@quantconnect.com")
client.connected

# Upload Asset Returns
# The returns file needs to have sufficient data to cover
# the backtest period of the algorithm in main.py and the in-sample
# backtest, which occurs before algorithm_start_date.
# Calculate the daily returns of the universe constituents.
tickers = [
    "TIP", "BWX", "EEM", "VGK", "IEF", "QQQ", "EWJ", "GLD",
    "VTI", "VNQ", "TLT", "RWX", "SPY", "DBC", "REM", "SCZ"
]
symbols = [qb.add_equity(ticker).symbol for ticker in tickers]
df = qb.history(
    symbols, datetime(2019, 1, 1), algorithm_end_date, Resolution.Daily
).close.unstack(0)

# Save the returns data into the Object Store.
df.rename(lambda x: x.split(' ')[0], axis='columns', inplace=True)
returns_file_name = "ETF_return_Test.csv"
```

```

returns_file_path = qb.object_store.get_file_path(returns_file_name)
df.pct_change().dropna().to_csv(returns_file_path)

# Upload the returns file to PredictNow.
message = client.upload_returns_file(returns_file_path)
print(message)

# List the return files you've uploaded.
return_files = client.list_returns_files()
','.join(return_files)

# Upload Constraints
# The constraints must contain a subset of the assets in the returns file. The CPO system only
# provides portfolio weights for assets that have constraints.
# Define the constraints.
constraints_by_symbol = {
    Symbol.create(ticker, SecurityType.EQUITY, Market.USA).ID: constraint
    for ticker, constraint in {
        "SPY": (0, 0.5),
        "QQQ": (0, 0.5),
        "VNQ": (0, 0.5)
    }.items()
}

# Create the constraints file.
content = "component, LB, UB"
for symbol, boundaries in constraints.items():
    content += f'\n{symbol},{boundaries[0]},{boundaries[1]}'

# Save the constraints file in the Object Store.
constraints_file_name = "ETF_constrain_Test.csv"
qb.object_store.save(constraints_file_name, content)

# Upload the constraints file to PredictNow.
constraint_file_path = qb.object_store.get_file_path(constraints_file_name)
message = client.upload_constraint_file(constraint_file_path)
print(message)

# List the constraint files you've uploaded.
constraint_files = client.list_constraint_files()
','.join(constraint_files)

# Define the Portfolio Parameters
portfolio_parameters = PortfolioParameters(
    name=f"Demo_Project_{datetime.now().strftime("%Y%m%d")}",
    returns_file=returns_file_name,
    constraint_file=constraints_file_name,
    #feature_file=feature_file_name,
    max_cash=1.0,
    rebalancing_period_unit="month",
    rebalancing_period=1,
    rebalance_on="first",
    training_data_size=3,
    evaluation_metric="sharpe"
)

# Run the In-Sample Backtest
# The in-sample period must end before the set_start_date in main.py.
# Since our algorithm does monthly rebalancing at the beginning
# of each month, the training_start_date argument should align with
# the start of the month and the training_end_date should be one day

```

```

# before the start date in main.py.
in_sample_result = client.run_in_sample_backtest(
    portfolio_parameters,
    training_start_date=datetime(2019, 1, 1),
    training_end_date=algorithm_start_date-timedelta(1),
    sampling_proportion=0.3,
    debug="debug"
)
print(in_sample_result)

def wait_for_backtest_to_finish(id_, sleep_seconds=60):
    job = client.get_job_for_id(id_)
    while job.status != "SUCCESS":
        job = client.get_job_for_id(id_)
        print(job.status)
        sleep(sleep_seconds)
    return job

job = wait_for_backtest_to_finish(in_sample_result.id)
print(job)
# Run the Out-Of-Sample Backtest
# The out-of-sample period should match the start and end dates
# of the algorithm main.py. It is important to keep the
# training_start_date parameters have the same format
# for in-sample and out-of-sample tests. For this example, we are
# working on a portfolio that takes monthly rebalance on the first
# market day of the month, so we will keep training_start_date
# to the 1st of the month in the out-of-sample test.

out_of_sample_result = client.run_out_of_sample_backtest(
    portfolio_parameters,
    training_start_date=algorithm_start_date,
    training_end_date=algorithm_end_date,
    debug="debug"
)
print(out_of_sample_result)

job = wait_for_backtest_to_finish(out_of_sample_result.id)
print(job)

# Get the Backtest Weights

# Let's get the portfolio weights from the preceding out-of-sample
# backtest. These are the weights you will use to run the LEAN
# algorithm in main.py. Save the portfolio weights into the Object
# Store so that you can load them in the algorithm.
weights_by_date = client.get_backtest_weights(
    portfolio_parameters,
    training_start_date=algorithm_start_date,
    training_end_date=algorithm_end_date,
    debug="debug"
)
print(weights_by_date)

# Save the weights into the Object Store.
qb.ObjectStore.Save("ETF_Weights_Test1.csv", json.dumps(weights_by_date))

```

CPO Code


```

# region imports
from AlgorithmImports import *

from QuantConnect.PredictNowNET import PredictNowClient
from QuantConnect.PredictNowNET.Models import *
from time import sleep
from datetime import datetime
# endregion

class PredictNowCPOAlgorithm(QCAlgorithm):
    """
    This algorithm demonstrates how to use PredictNow.ai to perform
    Conditional Portfolio Optimization (CPO). CPO utilizes the trailing
    asset returns and hundreds of other market features from PredictNow
    to determine weights that will maximize the future 1-month Sharpe
    ratio of the portfolio. The algorithm rebalances at the beginning
    of each month. To backtest this algorithm, first run the cells in
    the `research.ipynb` notebook.
    """

    def initialize(self):
        self.set_start_date(2020, 2, 1)
        self.set_end_date(2024, 4, 1)
        self.set_cash(100000)

        # Define the universe.
        tickers = [
            "TIP", "BWX", "EEM", "VGK", "IEF", "QQQ", "EWJ", "GLD",
            "VTI", "VNQ", "TLT", "RWX", "SPY", "DBC", "REM", "SCZ"
        ]
        self._symbols = [self.add_equity(ticker).symbol for ticker in tickers]

        if self.live_mode:
            # Connect to PredictNow.
            self._client = PredictNowClient("jared@quantconnect.com")
            if not self._client.connected:
                self.quit(f'Could not connect to PredictNow')
            # Define some parameters.
            self._in_sample_backtest_duration = timedelta(
                self.get_parameter("in_sample_days", 365)
            )
            self._out_of_sample_backtest_duration = timedelta(
                self.get_parameter("out_of_sample_days", 365)
            )
        else:
            # Read the weights file from Object Store.
            self._weights_by_date = pd.read_json(
                self.object_store.Read("ETF_Weights_Test1.csv")
            )
            # Schedule training and trading sessions.
            self.train(
                self.date_rules.month_start(self._symbols[0].symbol),
                self.time_rules.after_market_open(self._symbols[0].symbol, -10),
                self._rebalance
            )
            # Add a warm-up period to avoid errors on the first rebalance.
            self.set_warm_up(timedelta(7))

    def _rebalance(self):

```

```

# Don't trade during warm-up.
if self.is_warming_up:
    return

# In live mode, get the weights from PredictNow.
date = self.time.date()
if self.live_mode:
    self._get_live_weights(date)

# Create portfolio targets.
targets = []
for symbol, weight in self._weights_by_date[str(date)].items():
    self.log(f"Setting weight for {symbol.value} to {weight}")
    targets.append(PortfolioTarget(symbol, weight))

# Rebalance the portfolio.
self.set_holdings(targets, True)

def _get_live_weights(self, date):
    self.log(f"Loading live weights for {date}")

    # Upload the returns file to PredictNow.
    # Note: the history request includes an extra 2 months of data
    # so that we can move the start and end dates of the in-sample
    # and out-of-sample backtests so that they align with the start
    # and end of each month.
    self.debug(f"Uploading Returns file")
    returns_file_name = f"ETF_return_{str(date)}.csv"
    file_path = self.object_store.get_file_path(returns_file_name)
    returns = self.history(
        self._symbols,
        (self._in_sample_backtest_duration
         + self._out_of_sample_backtest_duration
         + timedelta(60)),
        Resolution.DAILY
    ).close.unstack(0).pct_change().dropna()
    returns.to_csv(file_path)
    self._client.upload_returns_file(file_path)
    self.debug(f"Uploaded file: {file_path}")

    # Create the asset weight constraints.
    content = "component, LB, UB"
    constraints_by_symbol = {
        Symbol.create(ticker, SecurityType.EQUITY, Market.USA).id: constraint
        for ticker, constraint in {
            "SPY": (0, 0.5),
            "QQQ": (0, 0.5),
            "VNO": (0, 0.5),
            "REM": (0, 0.33),
            "IEF": (0, 0.5),
            "TLT": (0, 0.5)
        }.items()
    }

    # Upload the constraints file to PredictNow.
    self.debug(f"Uploading Constraints file")
    for symbol, boundaries in constraints_by_symbol.items():
        content += f'\n{symbol},{boundaries[0]},{boundaries[1]}'
    constraints_file_name = f"ETF_constraint_{str(date)}.csv"
    self.object_store.save(constraints_file_name, content)
    file_path = self.object_store.get_file_path(constraints_file_name)
    self.debug(f"Uploaded file: {file_path}")

```

```

self._client.upload_constraint_file(file_path)

# Define the portfolio parameters.
portfolio_parameters = PortfolioParameters(
    name=f"Demo_Project_{str(date)}",
    returns_file=returns_file_name,
    constraint_file=constraints_file_name,
    max_cash=1.0,
    rebalancing_period_unit="month",
    rebalancing_period=1,
    rebalance_on="first",
    training_data_size=3,
    evaluation_metric="sharpe"
)

# Calculate the dates of the in- and out-of-sample backtests.
oos_start_date, oos_end_date = self._get_start_and_end_dates(
    date, self._out_of_sample_backtest_duration
)
is_start_date, is_end_date = self._get_start_and_end_dates(
    oos_start_date-timedelta(1), self._in_sample_backtest_duration
)

# Run the in-sample backtest.
self.debug("Running in-sample backtest")
in_sample_result = self._client.run_in_sample_backtest(
    portfolio_parameters,
    training_start_date=is_start_date,
    training_end_date=is_end_date,
    sampling_proportion=0.3,
    debug="debug"
)
in_sample_job = self._client.get_job_for_id(in_sample_result.id)

# Run the out-of-sample backtest.
self.debug("Running out-of-sample backtest")
out_of_sample_result = self._client.run_out_of_sample_backtest(
    portfolio_parameters,
    training_start_date=oos_start_date,
    training_end_date=oos_end_date,
    debug="debug"
)
out_of_sample_job = self._client.get_job_for_id(out_of_sample_result.id)

# Wait until the backtests finish running.
self.debug("Checking if the backtests are done")
while (in_sample_job.status != "SUCCESS" or
       out_of_sample_job.status != "SUCCESS"):
    in_sample_job = self._client.get_job_for_id(in_sample_result.id)
    out_of_sample_job = self._client.get_job_for_id(
        out_of_sample_result.id
    )
    self.debug(f"In Sample Job: {in_sample_job.status}")
    self.debug(f"Out of Sample Job: {out_of_sample_job.status}")
    sleep(60)

# Run the live prediction.
self.debug("Running Live Prediction")
exchange_hours = self.securities[self._symbols[0]].exchange.hours
live_prediction_result = self._client.run_live_prediction(
    portfolio_parameters,

```

```

        rebalance_date=date,
        next_rebalance_date=exchange_hours.get_next_market_open(
            Expiry.end_of_month(self.time), extended_market_hours=False
        ).date(),
        debug="debug"
    )
    live_job = self._client.get_job_for_id(live_prediction_result.id)

    # Wait until the live prediction job is done.
    self.debug("Checking Live prediction job status")
    while live_job.status != "SUCCESS":
        live_job = self._client.get_job_for_id(live_prediction_result.id)
        self.debug(f"Live Prediction status: {live_job.status}")
        sleep(60)

    # Get the prediction weights.
    self._weights_by_date = self._client.get_live_prediction_weights(
        portfolio_parameters,
        rebalance_date=date,
        debug="debug"
    )

def _get_start_and_end_dates(self, date, duration):
    start_date = end_date - duration
    start_date = datetime(start_date.year, start_date.month, 1)
    return start_date, end_date

```

Conclusion

In this chapter, we first demonstrated the power of machine learning to correct decisions made by an expert system, in this case an algorithmic Forex trading strategy, via our Corrective AI implementation. The key ingredients for success in Corrective AI are not so much the underlying machine-learning algorithm, but the proprietary input features one must engineer for a specific business process. Secondly, we demonstrated the success of Conditional Parameter Optimization (CPO) in optimizing the parameters of a business process, in this case an algorithmic ETF trading strategy. Thirdly, we demonstrated the superiority of Conditional Portfolio Optimization (also abbreviated as CPO) in optimizing resource allocation over conventional methods, in this case the capital allocation to portfolios of financial instruments.

It is intuitively obvious that the optimal solution to a problem depends on the environment in which it occurs, whether the problem is the optimal way to stock a retail shelf (back-to-school or holiday sales) or optimal asset allocation (risk-on or risk-off). Unfortunately, most conventional optimization methods cannot consider the environmental context, as it is often ill-defined and may involve hundreds of variables. However, machine-learning algorithms excel in dealing with big data inputs that may contain redundant and insignificant variables. Our CPO method leverages machine learning and Big Data to provide an optimal solution to many commercial problems such as portfolio optimization that adapts to the environment. We have demonstrated in multiple use cases that it can outperform conventional portfolio optimization methods and have shown an

example in tactical asset allocation where historical allocations were timely.

Definitions of Spread_EMA & Spread_VAR

$$\begin{aligned}
 \text{Spread_EMA}_0 &= \text{Spread}_0 \\
 \text{Spread_EMA}_{t+1} &= \frac{1}{2\text{lookback_period}} \text{Spread}_t + \frac{1}{2} + \left(1 - \frac{1}{2\text{lookback_period}}\right) \text{Spread_EMA}_t, \\
 \text{Spread_VAR}_1 &= (\text{Spread}_1 - \text{Spread}_0)^2, \\
 \text{Spread_VAR}_{t+1} &= \frac{1}{2\text{lookback_period}} (\text{Spread}_t + 1) - \text{Spread_EMA}_{t+1})^2 \\
 &\quad + \left(1 - \frac{1}{2\text{lookback_period}}\right) * \text{Spread_VAR}_t
 \end{aligned}$$